Arquitectura de Base de Datos

SaaS B2B de Datos

Documento de revisión técnica · Fecha: 2026-09-04

Ámbito del documento

Sustituye al modelo de datos descrito en «Arquitectura Base de Datos SaaS.pdf».

Fecha: 2026-09-04 · Estado: Propuesta de diseño · Tecnología base: Spring Boot, PostgreSQL, Python (IA)

1. Resumen ejecutivo

El diseño inicial proponía aislar los datos operativos de cada cliente mediante esquemas PostgreSQL independientes (**schema-per-tenant**). Tras el análisis, **este documento descarta ese enfoque** para los datos que el SaaS almacena en su base central y adopta un modelo de **tablas compartidas con `tenant_id` + Row-Level Security (RLS)**.

Justificación abreviada:

- Los datos sensibles del cliente (productos, ventas, pedidos) **no residen** en la base central del SaaS: viven en la base de datos externa del cliente, accesible únicamente en modo solo lectura.
- El aislamiento por esquema protegía un datos que no están en la base central, añadiendo en cambio costes operativos crecientes: migraciones ejecutadas N veces (una por schema), inflado del catálogo de objetos, riesgo de fuga por `search_path` residual en el pool de conexiones y mayor complejidad de provisionamiento en cada alta de cliente.
- Las tablas compartidas con `tenant_id` + RLS preservan el aislamiento lógico (garantizado por el propio motor de base de datos, no solo por el código) y escalan mejor a miles de clientes.

2. Decisiones de diseño

ID	Decisión
D1	Toda la información operativa del SaaS reside en el esquema public (sin esquemas por tenant).
D2	Las entidades compartidas Usuario y ConsultaHistorial incorporan tenant_id (FK a Tenant).
D3	El aislamiento se garantiza a nivel de base de datos mediante RLS de PostgreSQL.
D4	Las tablas compartidas usan UUID como PK (no SERIAL), evitando colisiones de identidad globales.
D5	Se elimina SET search_path. El contexto de tenant se propaga mediante la variable de sesión app.tenant_id.
D6	Los datos de negocio del cliente permanecen en su base externa; el SaaS nunca los copia en su base central.
D7	El límite de consultas y usuarios se valida por plan (Plan/Suscripcion), no por infraestructura.
D8	Los precios variables por antigüedad se modelan mediante tramos (plan_precios) y se preserva el precio aplicado en suscripcion_historial_precios.

3. Alcance de los almacenes de datos

Almacén	Contenido	Dueñ	Acceso

Base central del SaaS (PostgreSQL, SaaS)	Datos operativos: tenants, planes, suscripciones, conexiones, usuarios de la plataforma, historial de consultas IA	SaaS	Lectura/escritura
Base externa del cliente (PostgreSQL, solo lectura)	Datos de negocio: productos, ventas, pedidos, etc.	Cliente	Solo lectura mediante TenantDbConnection

El `SET search_path` del modelo anterior se utilizaba para enrutar las consultas de `usuarios` y `consultas_historial` hacia esquemas aislados dentro de la base central. En el nuevo modelo, esas consultas van a tablas compartidas filtradas por `tenant_id`, por lo que `search_path` deja de ser necesario.

4. Esquema general de la base central

```
public
├── tenants
├── planes
├── plan_precios
├── suscripciones
├── suscripcion_historial_precios
├── tenant_db_connections
├── usuarios              ← compartida · RLS por tenant_id
└── consultas_historial   ← compartida · RLS por tenant_id
```

5. Modelo de dominio (entidades)

Los métodos marcados como «común» (getters y setters) no se listan.

5.1 Tenant

Representa a la empresa cliente contratante del SaaS.

Campo	Tipo	Descripción
id	UUID	Clave primaria
nombreEmpresa	String	Nombre comercial
subdominioSlug	String	Identificador único (empresa_a)
fechaRegistro	LocalDateTime	Fecha de alta
estado	TenantEstado	ACTIVE, SUSPENDED, CANCELED

Relaciones: 1:N → `Suscripcion` · 1:N → `TenantDbConnection` · 1:N → `Usuario` · 1:N → `ConsultaHistorial`
Métodos de negocio:

```
public boolean estaActivo()
public void suspender()
public void cancelar()
public void reactivar()
```

5.2 Plan

Catálogo de planes comerciales (STARTER, GROWTH, ENTERPRISE).

Campo	Tipo	Descripción
id	UUID	Clave primaria
nombre	String	Nombre comercial
descripcion	String	Descripción y características

limiteConsultasMensuales	Integer	-1 = ilimitado
maxUsuarios	Integer	Máximo de usuarios por tenant
activo	Boolean	Disponible para contratación

Relaciones: 1:N → `PlanPrecio` · 1:N → `Suscripcion`

Métodos de negocio:

```
public boolean esIlimitado()
public boolean permiteUsuarios(Integer cantidad)
public boolean permiteConsultas(Integer cantidad)
public PlanPrecio getPrecioParaMes(Integer mesesContratados)
```

5.3 PlanPrecio

Tramos de precio por antigüedad de la contratación (precio fijo = un único tramo con `mesHasta` nulo).

Campo	Tipo	Descripción
id	UUID	Clave primaria
plan	Plan	FK plan_id
mesDesde	Integer	Inicio del tramo (1+)
mesHasta	Integer	Fin del tramo; null = indefinido
precio	BigDecimal	Precio mensual del tramo

Relaciones: N:1 → `Plan` · 1:N → `SuscripcionHistorialPrecio`

Métodos de negocio:

```
public boolean aplicaParaMes(Integer mesesContratados)
public boolean esTramoFinal()   // mesHasta == null
```

5.4 Suscripcion

Contrato vigente de un tenant con un plan.

Campo	Tipo	Descripción
id	UUID	Clave primaria
tenant	Tenant	FK tenant_id
plan	Plan	FK plan_id
fechaInicio	LocalDate	Fecha de contratación
estado	SuscripcionEstado	ACTIVE, CANCELED, SUSPENDED
proximaFacturacion	LocalDate	Próximo cobro

Relaciones: N:1 → `Tenant` · N:1 → `Plan` · 1:N → `SuscripcionHistorialPrecio`

Métodos de negocio:

```
public Integer getMesesContratados()   // Period.between(fechaInicio, now).toTotalMonths()
public boolean estaVigente()
public boolean requiereCambioDeTramo()
public void cancelar()
public void actualizarPlan(Plan nuevoPlan)
```

5.5 SuscripcionHistorialPrecio

Precio realmente aplicado por período; protege al cliente ante cambios de tarifa en `plan_precios`.

Campo	Tipo	Descripción
id	UUID	Clave primaria
suscripcion	Suscripcion	FK suscripcion_id
periodoInicio	LocalDate	Inicio del período
periodoFin	LocalDate	Fin del período
precioAplicado	BigDecimal	Precio cobrado en el período
planPrecio	PlanPrecio	FK al tramo usado (auditoría)

Relaciones: N:1 → `Suscripcion` · N:1 → `PlanPrecio`

Métodos de negocio:

```
public boolean estaVigente()          // hoy dentro del período
public Integer getDuracionDias()
public BigDecimal getPrecioDiario()
```

5.6 TenantDbConnection

Configuración de la conexión hacia la base del cliente (lectura o escritura según tier).

Campo	Tipo	Descripción
id	UUID	Clave primaria
tenant	Tenant	FK tenant_id
host	String	Servidor
puerto	Integer	Puerto (default 5432)
dbName	String	Nombre de la base externa
dbUser	String	Usuario con privilegios mínimos (solo SELECT)
dbPasswordEncrypted	String	Contraseña cifrada en reposo
tipoConexion	TipoConexion	EXTERNAL_DB (lectura), INTERNAL_SCHEMA (deprecado)
schemaName	String	Opcional; esquema objetivo dentro de la base externa

Relaciones: N:1 → `Tenant`

Métodos de negocio:

```
public boolean esConexionExterna()
public String getConnectionString()   // jdbc:postgresql://host:puerto/dbname
public void actualizarCredenciales(String user, String password)
public void marcarSoloLectura()
```

5.7 Usuario (tabla compartida + RLS)

Cuenta de acceso a la plataforma, pertenece a un tenant.

Campo	Tipo	Descripción
id	UUID	Clave primaria (global, evita colisiones entre tenants)
tenant	Tenant	FK tenant_id — filtro RLS
nombre	String	Nombre completo
email	String	Único dentro del tenant (unique(tenant_id, email))
rol	UsuarioRol	ADMIN, VIEWER
activo	Boolean	Habilita/deshabilita el acceso

Relaciones: N:1 → `Tenant` · 1:N → `ConsultaHistorial`

Métodos de negocio:

```
public boolean esAdmin()
public void cambiarRol(UsuarioRol nuevoRol)
public void deshabilitar()
```

5.8 ConsultaHistorial (tabla compartida + RLS)

Registro de cada consulta en lenguaje natural y su SQL generado.

Campo	Tipo	Descripción
id	UUID	Clave primaria
tenant	Tenant	FK tenant_id — filtro RLS
usuario	Usuario	FK usuario_id
preguntaUsuario	Text	Pregunta en lenguaje natural
querySqlGenerada	Text	SQL generado por IA
fechaConsulta	LocalDateTime	Momento de la consulta
exitosa	Boolean	Si la ejecución finalizó correctamente
registrosCount	Integer	Registros retornados
dbConnectionId	UUID	FK a TenantDbConnection utilizada (opcional)

Relaciones: N:1 → `Tenant` · N:1 → `Usuario`

Métodos de negocio:

```
public boolean fueExitosa()
public String getResumen()              // primeros 100 caracteres de la pregunta
public boolean superaLimite(Integer limiteConsultas)
```

5.9 Enumerados

```
public enum TenantEstado { ACTIVE, SUSPENDED, CANCELED }
public enum SuscripcionEstado { ACTIVE, CANCELED, SUSPENDED }
public enum TipoConexion { EXTERNAL_DB, INTERNAL_SCHEMA }
public enum UsuarioRol { ADMIN, VIEWER }
```

6. Aislamiento y seguridad (RLS)

El aislamiento entre tenants se garantiza en la capa de base de datos, no en la aplicación.

```
-- Habilitar RLS en las tablas compartidas
ALTER TABLE usuarios ENABLE ROW LEVEL SECURITY;
ALTER TABLE consultas_historial ENABLE ROW LEVEL SECURITY;

-- Obligar a que incluso el propietario de la tabla la respete
ALTER TABLE usuarios FORCE ROW LEVEL SECURITY;
ALTER TABLE consultas_historial FORCE ROW LEVEL SECURITY;

-- Políticas de aislamiento basadas en la variable de sesión
CREATE POLICY tenant_isolation_usuarios ON usuarios
  USING (tenant_id = current_setting('app.tenant_id')::uuid);

CREATE POLICY tenant_isolation_consultas ON consultas_historial
  USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

Nota: si un responsable de plataforma requiriera operar sobre todos los tenants (soporte, reporting interno), puede ejecutar consultas con `SET app.tenant_id = ...` vacío bajo un rol con privilegio de BYPASSRLS o una política adicional, controlado y auditado.

7. Flujo de aislamiento en la aplicación (Spring Boot)

```
Petición HTTP con header X-Tenant-ID: empresa_a
        │
        ▼
TenantInterceptor (HandlerInterceptor)
  → resuelve el Tenant a partir del header/subdominio
  → valida que exista y esté ACTIVO
  → ejecuta: SET app.tenant_id = '<uuid del tenant>'   (variable de sesión)
        │
        ▼
Controller → Service → Repository (JPA/Hibernate)
  → cualquier consulta a usuarios / consultas_historial
    sin WHERE tenant_id explícito
  → RLS filtra automáticamente por app.tenant_id
        │
        ▼
TenantInterceptor (afterCompletion)
  → limpia la variable de sesión y el contexto del hilo
```

Requisitos de implementación:

- `TenantContext` basado en `ThreadLocal` para propagar el tenant dentro del hilo de la petición.
- Cada conexión JDBC deja de depender de `search_path`; el aislamiento es lógico (RLS), no físico.
- La conexión a la base externa del cliente se resuelve desde `TenantDbConnection` y se marca `setReadOnly(true)`.

8. Comparación de modelos

Aspecto	Schema-per-tenant (descartado)	Tablas compartidas + tenant_id + RLS
Alta de cliente	CREATE SCHEMA + tablas + permisos	1 INSERT en tenants
Migraciones	Bucle sobre N esquemas	1 script
Pool de conexiones	Riesgo de fuga por search_path residual	Sin riesgo de esquema residual
Garantía de aislamiento	Física (redundante para datos operativos)	Lógica a nivel de motor (RLS)
Escalado a miles de tenants	Catálogo se infla (pg_class, pg_attribute)	Índices por (tenant_id, ...)
Reportes cross-tenant (admin)	Difícil	Consulta directa con filtro bajo rol controlado
Respaldo por tenant	pg_dump -n schema	Filtro lógico por tenant_id
Migración a otro motor	search_path varía por motor	RLS existe en PostgreSQL; en SQL Server depende de funcion LEVEL SECURITY

9. Escalabilidad y límites

- En PostgreSQL no existe un límite duro de esquemas, pero la cantidad de objetos (tablas) infla el catálogo y degrada planificación y autovacuum.
- Con tablas compartidas el crecimiento es de **filas**, no de objetos: los índices compuestos `(tenant_id, ...)` escalan igual que una tabla única y no degradan el catálogo.

• La arquitectura original era viable en el rango de miles de tenants; la nueva, al eliminar el coste por objeto, amplía ese rango y simplifica la operación diaria.

• Punto de atención: backups y restauración de un único tenant requieren filtros lógicos (o un proceso ETL) en lugar de `pg_dump -n`.

10. Plan de migración (desde el modelo schema-per-tenant)

Crear las tablas compartidas `usuarios` y `consultas_historial` añadiendo `tenant_id`.

Copiar cada fila de cada esquema `tenant_*` asignando el `tenant_id` correspondiente (relación `tenant_<slug>` ↔ `tenants.subdominio_slug`).

Reemplazar `id` serial por `UUID` (crear nueva columna, reasignar FKs, eliminar la antigua).

Activar RLS y crear las políticas.

Actualizar la entidad `TenantDbConnection` para eliminar la dependencia de `search_path`.

Desplegar la nueva versión de la aplicación sin cortar servicio (provisión en paralelo).

Validar aislamiento: ejecutar consultas cruzadas y verificar que RLS las bloquea.

Eliminar los esquemas `tenant_*` solo tras validación en entorno de prueba y respaldo.

11. Pendientes y riesgos

Tema	Estado
Definir rol «plataforma» con BYPASSRLS para soporte/reporting	Pendiente de decisión
Política de rotación y cifrado de db_password_encrypted	Pendiente
Batch de migración de datos (esquemas → tablas compartidas con UUID)	Pendiente
Indicadores de consumo (api_tokens_usados, limite_consultas) a nivel día frente a mes	Pendiente de especificación
Validación del SQL generado por IA (whitelist SELECT, sin ; ni --)	Definido en «Plan_SaaS_B2B_Datos.pdf» §6

Fin del documento. Este documento es la única referencia vigente para el modelo de datos de la base central del SaaS.